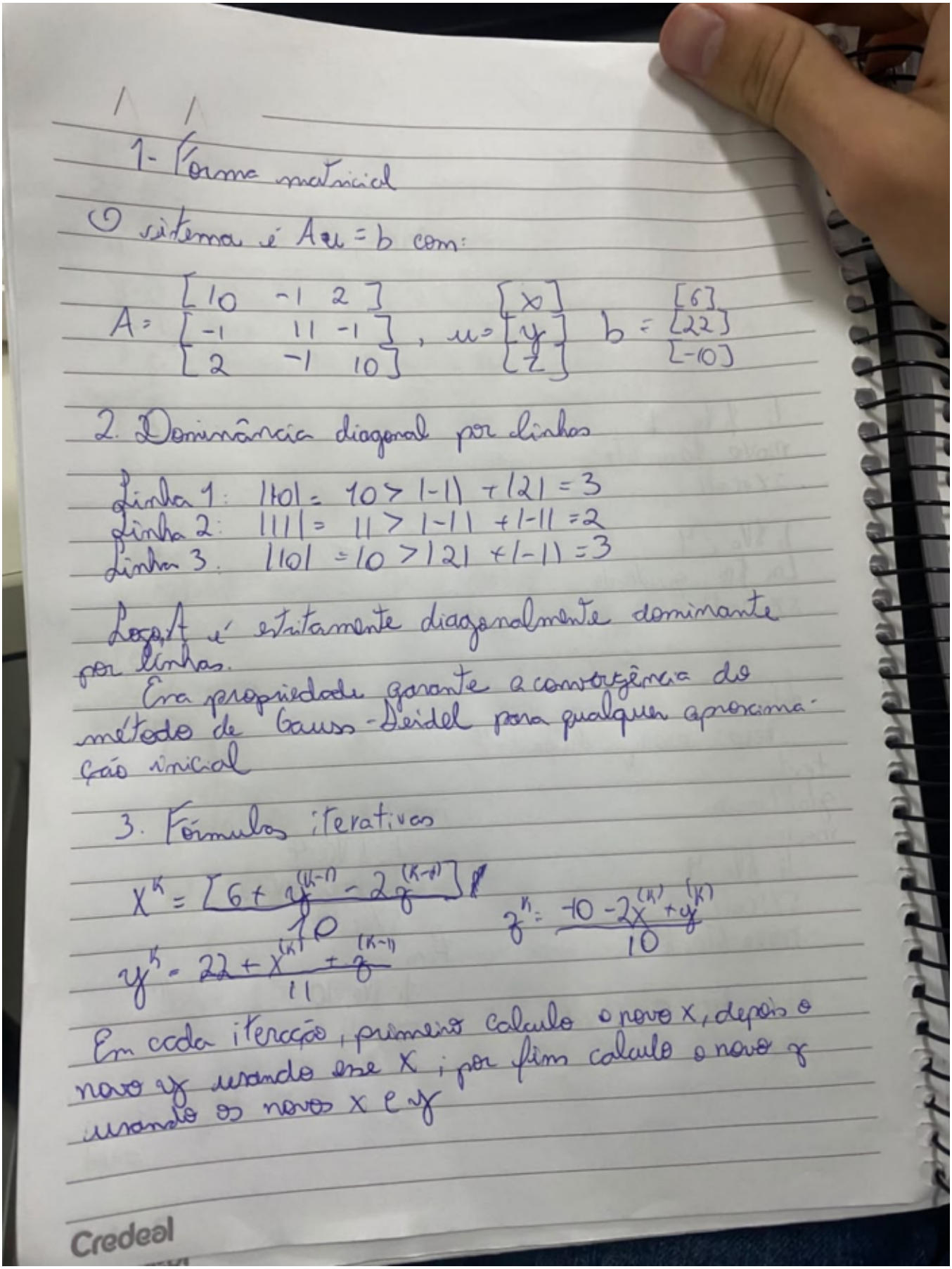
# Resolução manuscrita · questões 1 a 3



1- Forma matricial

O sistema é $Au = b$ com:

$$A = \begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix}, \quad u = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad b = \begin{bmatrix} 6 \\ 22 \\ -10 \end{bmatrix}$$

2. Dominância diagonal por linhas

Linha 1: $|10| = 10 > |-1| + |2| = 3$
Linha 2: $|11| = 11 > |-1| + |-1| = 2$
Linha 3: $|10| = 10 > |2| + |-1| = 3$

Logo, $A$ é estritamente diagonalmente dominante por linhas.

Essa propriedade garante a convergência do método de Gauss-Seidel para qualquer aproximação inicial

3. Fórmulas iterativas

$$x^{(k)} = \frac{6 + y^{(k-1)} - 2z^{(k-1)}}{10}$$

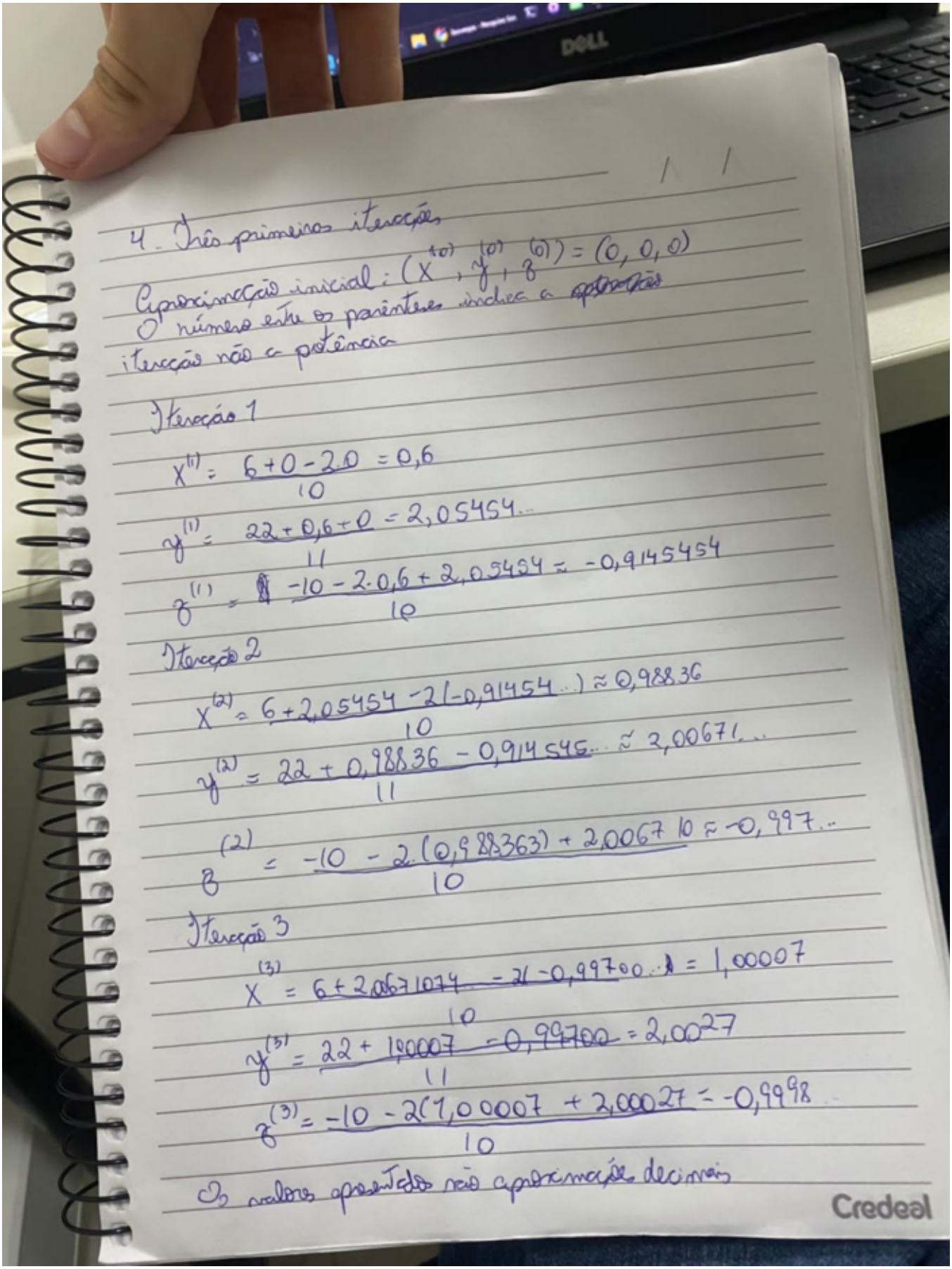
$$y^{(k)} = \frac{22 + x^{(k)} + z^{(k-1)}}{11}$$

$$z^{(k)} = \frac{-10 - 2x^{(k)} + y^{(k)}}{10}$$

Em cada iteração, primeiro calculo o novo $x$, depois o novo $y$ usando esse $x$, por fim calculo o novo $z$ usando os novos $x$ e $y$

Credeal

# Resolução manuscrita · questão 4




Correção: na iteração 3, y = 2,000279 e z = -0,999986 (ver cálculo digitado).

# Método de Gauss–Seidel

Prof.ª Raquel Lobosco | Sistemas lineares

## 1. Forma matricial A x = b

A = [[10, -1, 2], [-1, 11, -1], [2, -1, 10]]

x = [x, y, z]^T  b = [6, 22, -10]^T

## 2. Dominância diagonal por linhas

Linha 1: |10| = 10 > |-1| + |2| = 3

Linha 2: |11| = 11 > |-1| + |-1| = 2

Linha 3: |10| = 10 > |2| + |-1| = 3

A dominância estrita garante convergência para qualquer ponto inicial.

## 3. Fórmulas de atualização

x(k) = [6 + y(k-1) - 2 z(k-1)] / 10

y(k) = [22 + x(k) + z(k-1)] / 11

z(k) = [-10 - 2 x(k) + y(k)] / 10

Usamos x(k) e y(k) assim que são calculados.

## 4. Ponto inicial

(x(0), y(0), z(0)) = (0, 0, 0).

A seção seguinte detalha as três primeiras iterações.

# Três primeiras iterações

## Iteração 1

x(1) = (6 + 0 - 2·0)/10 = 0,6000000000

y(1) = (22 + 0,6 + 0)/11 = 2,0545454545

z(1) = (-10 - 2·0,6 + 2,0545454545)/10 = -0,9145454545

## Iteração 2

x(2) = [6 + 2,0545454545 - 2·(-0,9145454545)]/10

= 0,9883636364

y(2) = [22 + 0,9883636364 - 0,9145454545]/11

= 2,0067107438

z(2) = [-10 - 2·0,9883636364 + 2,0067107438]/10

= -0,9970016529

## Iteração 3

x(3) = [6 + 2,0067107438 - 2·(-0,9970016529)]/10

= 1,0000714050

y(3) = [22 + 1,0000714050 - 0,9970016529]/11

= 2,0002790684

z(3) = [-10 - 2·1,0000714050 + 2,0002790684]/10

= -0,9999863742

Valores exibidos arredondados; as iterações usam precisão completa.

# Código Python · método e critério

## Implementação essencial

```
def gauss_seidel(tol=1e-5, max_iter=100):
    x = y = z = 0.0
    linhas = [(0, x, y, z, None)]
    for k in range(1, max_iter + 1):
        anterior = (x, y, z)
        x = (6 + y - 2*z) / 10
        y = (22 + x + z) / 11
        z = (-10 - 2*x + y) / 10
        erro = max(abs(a-b) for a, b in
                   zip((x, y, z), anterior))
        linhas.append((k, x, y, z, erro))
        if erro < tol:
            return linhas
    raise RuntimeError('Sem convergência')
```

## Resultado

Critério: ||x(k) - x(k-1)|| infinito < 10^-5

Iterações realizadas: 6

Solução: (1,0000000269; 1,9999999762; -1,0000000078)

Erro da sexta iteração: 1,2626069386 × 10^-6

Solução exata para comparação: (1, 2, -1).

## Link para o código no Colab

O arquivo Gauss_Seidel_Lista.ipynb contém a versão completa para Colab.
https://colab.research.google.com/github/danielcordazzocorrea/calculo-numerico/blob/main/tarefa%20gauss/Gauss_Seidel_Lista.ipynb

# Tabela de iterações

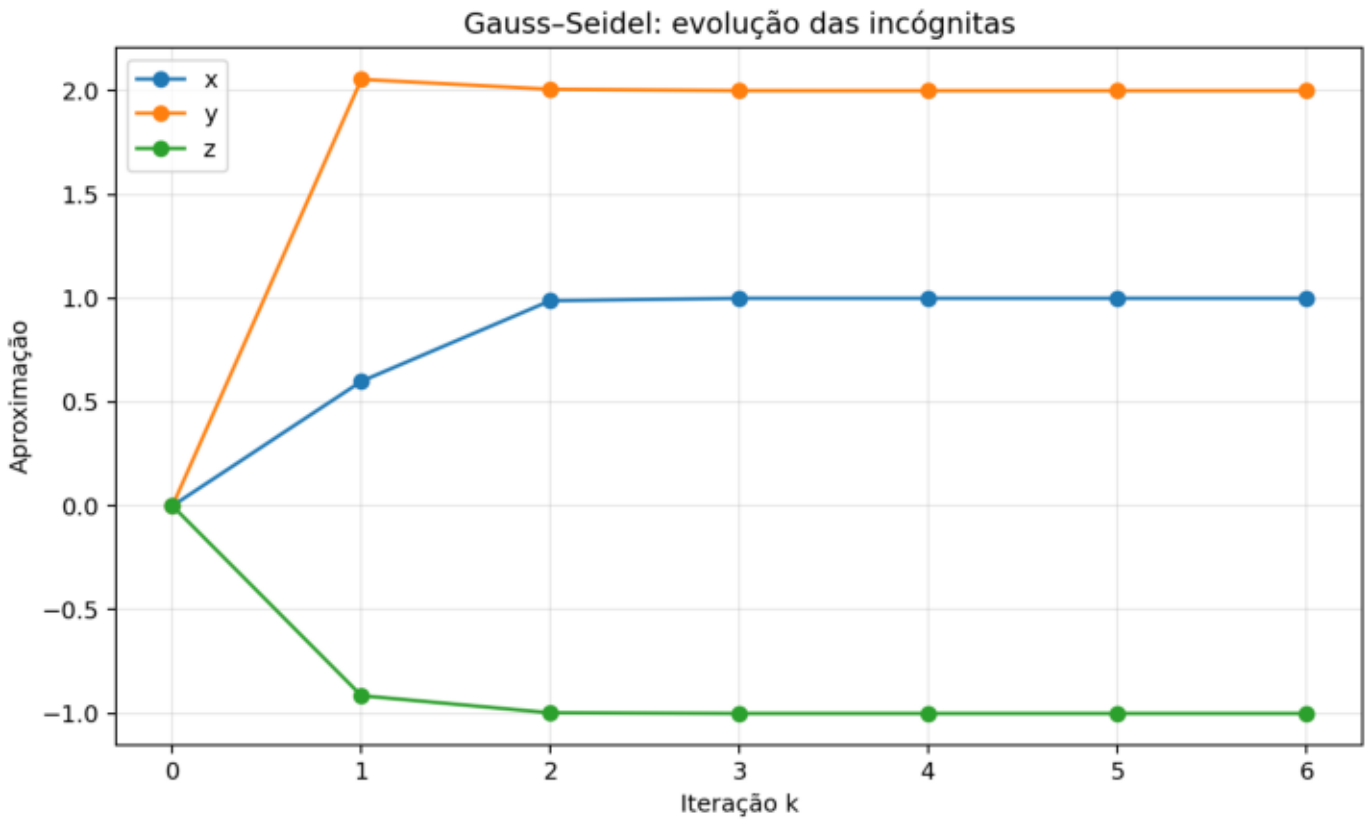
Critério de parada: erro entre iterações estritamente menor que 10^-5.

| k | x(k) | y(k) | z(k) | erro infinito |
|---|---|---|---|---|
| 0 | 0.0000000000 | 0.0000000000 | 0.0000000000 | — |
| 1 | 0.6000000000 | 2.0545454545 | -0.9145454545 | 2.05455e+00 |
| 2 | 0.9883636364 | 2.0067107438 | -0.9970016529 | 3.88364e-01 |
| 3 | 1.0000714050 | 2.0002790684 | -0.9999863742 | 1.17078e-02 |
| 4 | 1.0000251817 | 2.0000035280 | -1.0000046835 | 2.75540e-04 |
| 5 | 1.0000012895 | 1.9999996915 | -1.0000002888 | 2.38922e-05 |
| 6 | 1.0000000269 | 1.9999999762 | -1.0000000078 | 1.26261e-06 |

Em k=5, o erro ainda é maior que a tolerância.

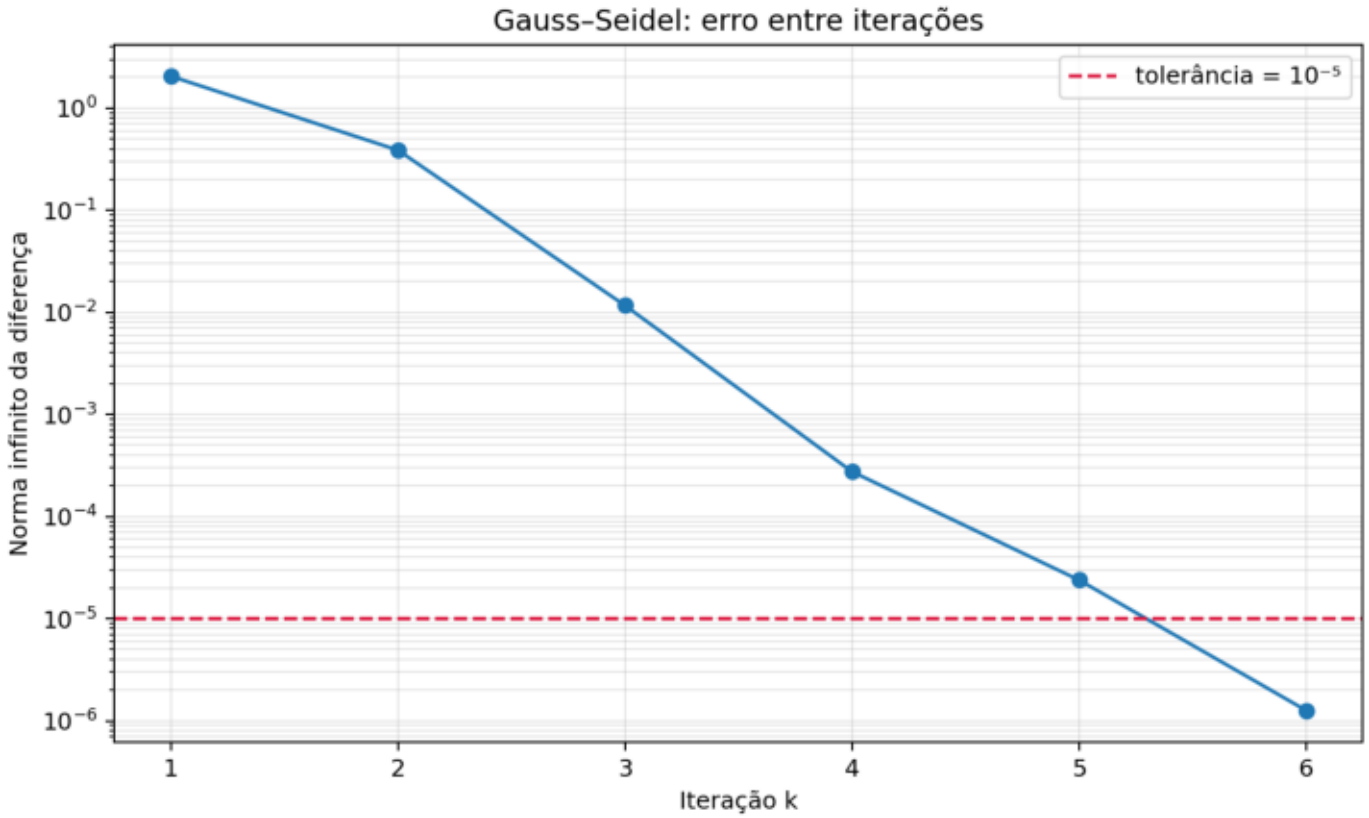
Em k=6, o critério é atendido e o processo para.

# Evolução das aproximações




Fonte: execução do código Python da atividade.

# Erro entre iterações em escala logarítmica




Fonte: execução do código Python da atividade.